

Guide de Preparation

Entretien System Design

30 sujets prioritaires – Profil Data / Streaming / IA

Framework d'entretien, estimations, architectures et trade-offs

Kafka – Spark Streaming – RAG – Fraud Detection – Recommendation
Uber Dispatch – Netflix – Google Drive – Data Lake – Feature Store

*L'objectif n'est pas de memoriser des solutions, mais de maitriser un raisonnement reutilisable :
exigences, estimation, architecture, trade-offs.*

Table des matières

1	Le framework attendu pendant l'entretien	2
2	Streaming et Data Engineering	3
2.1	Design Kafka (plateforme de streaming d'évenements)	3
2.2	Design Spark Streaming Pipeline	3
2.3	Design Real-Time Analytics Platform	4
2.4	Design CDC Pipeline (Change Data Capture)	5
2.5	Design Data Lake	5
2.6	Design ETL Platform	6
2.7	Design Airflow Scheduler / Orchestrateur de pipelines	6
3	IA generative et systemes intelligents	8
3.1	Design Recommendation Engine	8
3.2	Design Search Engine	8
3.3	Design Vector Database	9
3.4	Design RAG System (Retrieval-Augmented Generation)	9
3.5	Design ChatGPT / AI Assistant conversationnel	10
3.6	Design Multi-Agent System (IA)	11
3.7	Design ML Feature Store	11
4	Systemes metier critiques	13
4.1	Design Fraud Detection System (temps-reel)	13
4.2	Design Payment Gateway	13
4.3	Design Uber Dispatch / Ride Sharing Backend	14
4.4	Design Netflix Streaming (video a la demande)	15
4.5	Design Distributed Cache (type Redis)	15
5	Produits grand public a forte echelle	17
5.1	Design URL Shortener (TinyURL / Bitly)	17
5.2	Design Rate Limiter	17
5.3	Design Notification Service	18
5.4	Design WhatsApp / Messenger (messagerie instantanee)	18
5.5	Design Instagram Feed / Facebook News Feed	19
5.6	Design YouTube (plateforme video)	20
5.7	Design Google Drive / Dropbox (stockage et synchronisation de fichiers)	20
5.8	Design Reddit (forum de discussion avec vote)	21
5.9	Design Slack / Discord (messagerie d'equipe et communautes)	22
6	Aide-memoire final	23

1. Le framework attendu pendant l'entretien

A retenir Le principe numero un

La communauté (Reddit, Exponent, Gaurav Sen, ByteByteGo) converge sur un seul message : **ne memorisez pas des solutions toutes faites**. Un intervieweur reconnaît immédiatement un candidat qui recite une architecture apprise par cœur face à un qui raisonne. Ce qui est évalué : la maîtrise des *compromis* (trade-offs), la capacité à partir des exigences (et non d'une techno favorite), et l'aptitude à expliquer son raisonnement à voix haute.

Les entretiens de system design suivent quasi-systématiquement cette structure en 12 étapes. Utilisez-la comme checklist mentale pour chaque sujet, y compris pendant votre révision.

1. **Clarifier les besoins** – poser des questions avant de concevoir quoi que ce soit (périmètre, échelle, contraintes).
2. **Exigences fonctionnelles** – que doit *faire* le système (les cas d'usage concrets, priorisés).
3. **Exigences non fonctionnelles** – disponibilité, latence, cohérence, durabilité, scalabilité.
4. **Estimation du trafic** – QPS (requêtes/seconde), volume de stockage, bande passante (back-of-envelope).
5. **Architecture de haut niveau** – schéma en boîtes : client, API, services, stockage, cache, files.
6. **Détail des composants** – zoomer sur 1 ou 2 composants critiques (souvent celui que l'intervieweur relance).
7. **Choix des bases de données** – SQL vs NoSQL, et pourquoi, en fonction du modèle d'accès aux données.
8. **Stratégie de cache** – quoi cacher, où (client, CDN, application, base), quelle politique d'éviction.
9. **Files de messages** – où et pourquoi découpler des composants (Kafka, SQS, RabbitMQ).
10. **Scalabilité** – scaling horizontal vs vertical, partitionnement/sharding, load balancing.
11. **Haute disponibilité** – réplication, failover, redondance multi-zone/multi-region.
12. **Goulots d'étranglement** – les identifier proactivement et proposer des améliorations avant qu'on vous les demande.

Règles de raisonnement transversales

CAP theorem : en cas de partition réseau, choisir entre Cohérence (Consistency) et Disponibilité (Availability) – jamais les deux simultanément. **SQL vs NoSQL** : SQL si les relations et transactions ACID sont centrales (paiements, inventaire) ; NoSQL si le volume/vitesse d'écriture prime et que le modèle d'accès est simple et prévisible (feed, logs, sessions). **Cache** : accélère les lectures répétées mais introduit un risque de données périmées (staleness) – toujours préciser la stratégie d'invalidation. **File de messages** : découple producteur et consommateur, absorbe les pics de charge, mais ajoute de la latence et de la complexité opérationnelle (monitoring du lag, gestion des doublons).

Piège classique

Le piège le plus fréquent est de foncer directement sur l'architecture avant d'avoir clarifié les exigences. Un candidat qui dessine immédiatement "Kafka + Spark + Redis" sans avoir demandé le volume de trafic ou la tolérance à la latence donne l'impression de suivre un template plutôt que de résoudre le problème posé. Passez toujours au moins 3-5 minutes sur les étapes 1 à 4 avant de dessiner quoi que ce soit.

2. Streaming et Data Engineering

2.1 Design Kafka (plateforme de streaming d'évenements)

Exigences

Fonctionnel : publier des evenements (producers), les consommer (consumers) avec plusieurs consommateurs independants sur le meme flux, conserver un historique rejouable. **Non-fonctionnel** : tres haut debit (millions de messages/sec), faible latence, durabilite (pas de perte de donnees), ordre garanti au moins par cle.

Architecture haut niveau

Log distribue immuable : chaque *topic* est divise en *partitions*, chacune etant un log ordonne et append-only reple sur plusieurs *brokers* (facteur de replication typique = 3). Un *leader* par partition gere les ecritures, les *followers* repliquent en continu. Les producers ecrivent via une cle de partitionnement (hash de la cle → partition). Les consumers, organises en *consumer groups*, se repartissent les partitions et memorisent leur *offset* de lecture pour permettre une reprise fiable.

Trade-offs cles

Pourquoi plusieurs partitions ? Parallelisme horizontal en ecriture et en lecture – sans elles, un topic serait limite au debit d'un seul broker. **Comment eviter la perte de messages ?** `acks=all` cote producer (attendre l'accuse de reception de toutes les replicas in-sync avant de considerer l'ecriture reussie) + facteur de replication ≥ 3 + `min.insync.replicas` configure. **Exactly-once ?** Producer idempotent (evite les doublons dus aux retries) combine a des transactions Kafka (`isolation.level=read_committed`), ou idempotence applicative cote consumer. **Ordre des messages ?** Garanti uniquement *au sein* d'une partition – router les evenements d'une meme entite vers la meme partition via sa cle. **Replication ?** Facteur N (souvent 3) reparti sur des racks/AZ differents pour survivre a une panne de broker ou de zone. **Dimensionnement ?** Nombre de partitions dimensionne pour le parallelisme cible (nb consumers max utile = nb partitions), retention basee sur l'espace disque disponible et le besoin de rejouabilite.

2.2 Design Spark Streaming Pipeline

Exigences

Fonctionnel : ingerer un flux continu d'évenements, appliquer des transformations/agregations en quasi temps-reel, ecrire le resultat vers un sink (data lake, base, dashboard). **Non-fonctionnel** : faible latence (secondes), tolerance aux pannes avec reprise sans perte ni doublon, passage a l'échelle avec le volume d'entree.

Architecture haut niveau

Source (Kafka) → **Structured Streaming** (micro-batches ou mode continu) → transformations (agregations par fenetre, jointures stream-stream ou stream-table) → **sink** (Delta Lake, base de series temporelles, dashboard). Le moteur decoupe le flux en micro-batches traites successivement, chacun *checkpointe* (offsets + etat intermediaire) sur un stockage durable pour permettre une reprise exacte apres panne.

Trade-offs clés

Batch vs Streaming ? Streaming si la fraîcheur des résultats en quasi temps-reel a une valeur métier directe (fraude, monitoring) ; batch sinon (moins coûteux, plus simple à déboguer, suffisant pour du reporting quotidien). **Spark vs Flink ?** Spark Structured Streaming s'intègre naturellement si l'écosystème batch existant est déjà Spark (code unifié batch/streaming, écosystème Delta Lake) ; Flink est conçu nativement pour le streaming avec une latence plus faible et un modèle d'état plus riche – pertinent si la latence sub-seconde est critique. **Watermark ?** Définit jusqu'à quel point on tolère des événements en retard avant de fermer définitivement une fenêtre temporelle – compromis explicite entre complétude des résultats et latence de mise à disposition. **Window ?** Fenêtres tumbling (sans chevauchement), sliding (chevauchantes) ou session (basées sur l'inactivité) selon le besoin d'aggrégation. **Checkpoint ?** Sauvegarde périodique de l'état et des offsets pour permettre une reprise exacte après panne, sans recalculer depuis le début ni perdre l'état accumulé. **Backpressure ?** Mécanisme qui ralentit l'ingestion quand le traitement ne suit pas le débit d'entrée, évitant une accumulation incontrôlée en mémoire. **State management ?** L'état des agrégations en cours (ex : compteur par fenêtre) est maintenu dans un *state store* (souvent RocksDB local + checkpoint distant) pour survivre aux redémarrages de tâche.

2.3 Design Real-Time Analytics Platform**Exigences**

Fonctionnel : ingérer des événements en continu (clics, transactions), calculer des métriques agrégées en temps quasi-reel, exposer ces métriques via des dashboards avec rafraîchissement automatique. **Non-fonctionnel** : latence bout-en-bout de quelques secondes, haute disponibilité du pipeline d'ingestion, capacité à absorber des pics de trafic (ex : soldes, événement viral).

Estimation (back-of-envelope)

Exemple de raisonnement (à adapter aux chiffres donnés par l'intervieweur) : 100M événements/jour $\approx$ 1 150 événements/sec en moyenne, avec des pics à 5–10x en heure de pointe $\Rightarrow$ dimensionner l'ingestion pour $\sim$ 10 000 événements/sec. Un événement de $\sim$ 1 Ko $\Rightarrow$ $\sim$ 100 Go/jour de données brutes à stocker/retenir.

Architecture haut niveau

Sources $\rightarrow$ **Kafka** (ingestion, buffer) $\rightarrow$ **Spark/Flink Streaming** (aggrégations par fenêtre glissante) $\rightarrow$ double écriture : (a) **base série-temporelle / OLAP** (ex : Druid, ClickHouse, Pinot) pour les requêtes de dashboard à faible latence, (b) **data lake** (Delta/Iceberg) pour l'historique complet et le retraitement. Un service API sert les dashboards en lisant l'OLAP store, jamais directement le flux brut.

Trade-offs clés

Le choix clé est la base servant les dashboards : une base OLAP spécialisée (Druid, ClickHouse) plutôt qu'un entrepôt batch classique, car elle est conçue pour l'ingestion continue combinée à des requêtes d'aggrégation sub-seconde. Compromis assumé : moins flexible pour des requêtes ad hoc complexes qu'un entrepôt SQL général, mais bien plus rapide pour le pattern "dashboard temps-reel" qui domine le cas d'usage.

2.4 Design CDC Pipeline (Change Data Capture)

Exigences

Fonctionnel : capturer chaque insertion/modification/suppression dans une base operationnelle (ex : PostgreSQL) et la propager vers des systemes downstream (data lake, cache, moteur de recherche) sans impacter les performances de la base source. **Non-fonctionnel** : faible latence de propagation, aucune perte d'évenement, ordre preserve par entite.

Architecture haut niveau

Base source (avec write-ahead log / binlog activé) → **connecteur CDC** (ex : Debezium) qui lit le log de transactions directement (pas de polling par requetes SQL couteuses) → **Kafka** (un topic par table, evenements insert/update/delete) → consommateurs multiples en aval (indexation search, invalidation de cache, synchronisation vers le data lake).

Trade-offs clés

Pourquoi CDC plutot qu'un polling periodique ? Le polling (ex : `SELECT * WHERE updated_at > last_run`) rate les suppressions, cree une charge repetees sur la base, et introduit une latence liee a la frequence de polling. Le CDC base sur le log de transactions capture *tous* les changements (y compris deletes) en quasi temps-reel, avec un impact minimal sur la base source puisqu'il lit un log deja ecrit pour la durabilite, pas les tables elles-memes. Compromis : complexite operationnelle superieure (gerer le connecteur, les schema changes) et couplage plus fort au moteur de base de donnees source.

2.5 Design Data Lake

Exigences

Fonctionnel : centraliser des donnees brutes de sources heterogenes (bases, evenements, fichiers), les rendre exploitables par plusieurs equipes (data engineering, analytics, ML) avec des besoins et des outils differents. **Non-fonctionnel** : cout de stockage maitrise a grande echelle, gouvernance et securite (qui peut acceder a quoi), fiabilite des ecritures concurrentes.

Architecture haut niveau

Stockage objet (S3/ADLS/GCS) organise en couches **Bronze** (brut, tel qu'ingere), **Silver** (nettoye, deduplique, jointures de base), **Gold** (agreges, modelise pour la consommation metier). Un format de table transactionnel (Delta Lake / Iceberg / Hudi) est pose par-dessus les fichiers Parquet pour apporter des garanties ACID. Un catalogue de metadonnees (ex : Glue Data Catalog, Hive Metastore) rend les donnees decouvrables et interrogeables via SQL par plusieurs moteurs (Spark, Trino, Athena).

Trade-offs clés

Pourquoi un data lake plutot qu'un entrepot classique uniquement ? Le data lake accepte des donnees non structurees/semi-structurees a bas cout de stockage et differe le schema a la lecture (*schema-on-read*), utile quand les besoins d'analyse ne sont pas encore tous connus a l'ingestion – contrairement a un entrepot qui impose un schema rigide a l'ecriture. Compromis : sans discipline (couches Bronze/Silver/Gold, catalogue, format transactionnel), un data lake degeneere facilement en "data swamp" difficile a gouverner et a interroger de maniere fiable.

2.6 Design ETL Platform

Exigences

Fonctionnel : extraire des données de sources multiples (APIs, bases, fichiers), les transformer selon des règles métier, les charger dans une destination analytique, de manière planifiée et fiable. **Non-fonctionnel** : reprise automatique sur échec, visibilité sur l'état de chaque pipeline, idempotence (rejouable sans effet de bord), respect de SLA (données prêtes avant une heure donnée).

Architecture haut niveau

Orchestrateur (Airflow) déclenche des DAGs selon un planning ou un événement. **Extraction** : connecteurs vers les sources (API REST, CDC, exports batch). **Transformation** : privilégier l'**ELT** – charger brut dans l'entrepôt/lake puis transformer en SQL déclaratif avec **dbt** (versionne, teste, documente) plutôt qu'un moteur de transformation externe coûteux à maintenir. **Chargement** : écriture idempotente (MERGE/UPSERT par clé métier ou overwrite de partition) dans la couche cible.

Trade-offs clés

Le choix ETL vs ELT est le trade-off central : ELT tire parti de la puissance de calcul élastique des entrepôts cloud modernes et conserve les données brutes pour re-transformer facilement en cas d'évolution des règles métier, au prix d'une consommation de calcul dans l'entrepôt lui-même (facturation à la requête pour BigQuery/Snowflake). ETL garde l'entrepôt "propre" mais nécessite de maintenir un moteur de transformation séparé et de re-extraire pour changer une règle historique.

2.7 Design Airflow Scheduler / Orchestrateur de pipelines

Exigences

Fonctionnel : définir des pipelines comme des graphes de tâches avec dépendances explicites, les déclencher selon un planning (cron) ou un événement, gérer les reprises sur échec. **Non-fonctionnel** : scalabilité horizontale de l'exécution (des milliers de tâches concurrentes), visibilité opérationnelle (UI, alerting), tolérance aux pannes du scheduler lui-même.

Architecture haut niveau

Scheduler : évalue en continu les DAGs et décide quelles tâches sont prêtes à s'exécuter (dépendances satisfaites, planning atteint). **Metastore** (base relationnelle) : état de toutes les exécutions passées/en cours. **Executor** : distribue l'exécution réelle des tâches (**CeleryExecutor** via une file de messages + workers, ou **KubernetesExecutor** qui lance un pod éphémère par tâche pour une isolation et un scaling élastique complets). **Webserver** : UI de supervision.

Trade-offs clés

CeleryExecutor vs KubernetesExecutor ? Celery mutualise un pool de workers pré-démarrés (latence de démarrage de tâche plus faible, mais ressources réservées même inactives). Kubernetes lance un pod par tâche (isolation totale des dépendances/ressources par tâche, scaling à zéro possible), au prix d'une latence de démarrage de pod à chaque exécution. **Comment le scheduler passe-t-il à l'échelle ?** En le rendant lui-même actif-actif (plu-

plusieurs instances de scheduler évaluant les DAGs en parallèle avec verrouillage optimiste sur le metastore) et en déportant l'exécution réelle sur l'executor, jamais sur le scheduler.

3. IA generative et systemes intelligents

3.1 Design Recommendation Engine

Exigences

Fonctionnel : suggerer des items (produits, videos, posts) pertinents pour un utilisateur donne, en tenant compte de son historique et du contexte courant. **Non-fonctionnel** : latence de servir une recommandation en dizaines de millisecondes, fraicheur (integrer les signaux recents), passage a l'echelle sur des catalogues de millions d'items.

Architecture haut niveau

Architecture en deux etages, standard dans l'industrie : **Candidate Generation** (retrieval) – reduire rapidement des millions d'items a quelques centaines de candidats plausibles, souvent via un modele de similarite d'embeddings (recherche vectorielle approximee) ou du filtrage collaboratif precalcule. **Ranking** – un modele plus lourd (souvent un reseau de neurones) reordonne precisement ces quelques centaines de candidats en tenant compte de features riches (contexte, recence, profil utilisateur), calculees a la volee via un **feature store**. Les recommandations candidates sont souvent precalculees en batch/streaming et cachees, le ranking final se faisant en ligne.

Trade-offs clés

Le compromis central est retrieval (rapide, approximatif, sur tout le catalogue) vs ranking (lent, precis, sur un petit sous-ensemble) – appliquer directement un modele complexe sur des millions d'items en ligne serait bien trop lent. Autre trade-off : filtrage collaboratif (necessite un historique d'interactions, souffre du *cold start* sur les nouveaux items/utilisateurs) vs approche basee sur le contenu (fonctionne des le premier item mais capture moins bien les gouts implicites) – les systemes de production combinent generalement les deux.

3.2 Design Search Engine

Exigences

Fonctionnel : indexer un grand corpus de documents, repondre a une requete texte libre avec les resultats les plus pertinents, classes par pertinence. **Non-fonctionnel** : latence de recherche sous 100ms, index mis a jour en continu sans interrompre les recherches, scalabilite horizontale sur des milliards de documents.

Architecture haut niveau

Pipeline d'indexation : documents → analyse/tokenisation → construction d'un **index inverse** (mot → liste des documents le contenant, avec position et frequence) → index distribue et **sharde** (ex : Elasticsearch/OpenSearch) avec replication pour la disponibilite. **Pipeline de requete** : la requete est tokenisee de la meme facon, envoyee en parallele a tous les shards pertinents (*scatter-gather*), chaque shard retourne ses meilleurs resultats locaux scores (souvent via BM25), un noeud coordinateur fusionne et retreie le resultat global.

Trade-offs clés

Index inverse vs scan lineaire : l'index inverse rend la recherche sous-lineaire par rapport au nombre de documents (on ne consulte que les documents contenant les termes cherches), indispensable a l'echelle du Web/d'un gros catalogue. **Sharding** : necessaire quand l'index depasse la capacite d'une seule machine, mais chaque requete devient une operation distribuee scatter-gather – la latence totale est bornee par le shard le plus lent, ce qui impose une attention particuliere a l'equilibrage de charge entre shards.

3.3 Design Vector Database

Exigences

Fonctionnel : stocker des vecteurs d'embeddings de haute dimension (ex : 768 ou 1536 dimensions), retrouver les k vecteurs les plus proches d'un vecteur requete (recherche de similarite). **Non-fonctionnel** : latence de recherche faible meme sur des centaines de millions de vecteurs, mises a jour incrementales de l'index sans reconstruction complete.

Architecture haut niveau

Le coeur du systeme est un **index de recherche approximative des plus proches voisins (ANN)** – une recherche exacte (calculer la distance a tous les vecteurs) est bien trop couteuse a grande echelle. Algorithmes courants : **HNSW** (graphe hierarchique navigable, tres bon compromis vitesse/precision, standard de facto), ou **IVF** (partitionnement de l'espace vectoriel en clusters, on ne cherche que dans les clusters les plus proches du vecteur requete). Les vecteurs sont sharde s par volume, avec des metadonnees associees permettant un filtrage hybride (ex : "similaire a X ET categorie = Y").

Trade-offs clés

ANN vs recherche exacte : l'ANN sacrifie une garantie de trouver *exactement* les k plus proches voisins contre un gain de plusieurs ordres de grandeur en vitesse – acceptable car en pratique, un resultat "presque le plus proche" est indiscernable metier d'un resultat exact. **HNSW vs IVF** : HNSW offre generalement une meilleure precision a latence egale mais consomme plus de memoire (graphe garde en RAM); IVF est plus economique en memoire et plus simple a re-entrainer/re-partitionner mais moins precis a rappel egal.

3.4 Design RAG System (Retrieval-Augmented Generation)

Exigences

Fonctionnel : repondre a une question utilisateur en s'appuyant sur un corpus de documents specifique (pas seulement les connaissances parametriques du LLM), avec citation des sources. **Non-fonctionnel** : latence bout-en-bout acceptable pour un usage conversationnel (quelques secondes), fraicheur du corpus indexe, limiter les hallucinations en ancrant la reponse dans les documents recuperes.

Architecture haut niveau

Pipeline d'ingestion : documents → découpage en chunks (avec chevauchement) → modèle d'embedding → stockage dans une vector database, avec les métadonnées (source, date). **Pipeline de requête** : question utilisateur → embedding de la question → recherche de similarité (top- k chunks pertinents, éventuellement combinée à une recherche lexicale classique = *hybrid search*) → les chunks récupérés sont injectés comme contexte dans le prompt envoyé au LLM → génération de la réponse finale, avec citation des chunks sources.

Trade-offs clés

Taille des chunks : des chunks trop grands diluent la pertinence de la recherche vectorielle (un chunk contient plusieurs sujets, son embedding devient un "moyennage" flou) ; des chunks trop petits perdent le contexte nécessaire à une réponse cohérente – un compromis empirique typique se situe entre quelques centaines de tokens, avec chevauchement pour ne pas couper une idée en deux. **Recherche vectorielle seule vs hybride** : la recherche vectorielle capture la similarité sémantique mais rate parfois des correspondances exactes (noms propres, codes, acronymes) qu'une recherche lexicale (BM25) trouve facilement – combiner les deux (*hybrid search*) améliore généralement le rappel. **Réduire les hallucinations** : forcer le prompt à n'utiliser que le contexte fourni ("répondre uniquement à partir des documents ci-dessous, dire si l'information est absente"), et exposer les sources pour permettre une vérification humaine.

3.5 Design ChatGPT / AI Assistant conversationnel

Exigences

Fonctionnel : conversation multi-tour avec mémoire du contexte de la session, génération de réponse en streaming (token par token), gestion de sessions multiples par utilisateur. **Non-fonctionnel** : faible latence du premier token (*time-to-first-token*), scalabilité du service d'inférence sous forte charge concurrente, coût d'inférence maîtrisé (le calcul GPU est cher).

Architecture haut niveau

Client (streaming via WebSocket ou Server-Sent Events pour afficher la réponse au fur et à mesure) → **API Gateway** (authentification, rate limiting) → **Conversation Service** (récupère l'historique depuis un store de sessions, construit le prompt complet) → **Inference Service** (cluster de GPUs servant le LLM, avec batching dynamique des requêtes concurrentes pour maximiser l'utilisation GPU) → réponse streamée de retour au client, historique persiste après completion.

Trade-offs clés

Pourquoi le streaming token-par-token ? Le temps de génération complet d'une longue réponse peut dépasser plusieurs secondes ; streamer chaque token dès qu'il est généré améliore drastiquement la latence perçue, même si la latence totale est identique. **Gestion de la mémoire de conversation** : garder tout l'historique dans le prompt dégrade la latence et le coût (le coût d'inférence croît avec la longueur du prompt) – des stratégies de résumé progressif ou de troncature de l'historique ancien sont nécessaires au-delà d'un certain nombre de tours. **Batching dynamique côté inférence** : grouper plusieurs requêtes utilisateur concurrentes dans un seul batch GPU améliore le débit global, au prix d'une latence individuelle légèrement accrue pour les requêtes arrivées en premier dans le batch – compromis débit/latence classique.

3.6 Design Multi-Agent System (IA)

Exigences

Fonctionnel : decomposer une tache complexe en sous-taches confiees a des agents specialises (ex : agent de recherche, agent de code, agent de validation) qui collaborent pour produire un resultat final. **Non-fonctionnel** : traçabilité complete du raisonnement (audit des decisions de chaque agent), robustesse face a l'echec d'un agent individuel, maitrise du cout cumule (chaque appel agent = un appel LLM facture).

Architecture haut niveau

Orchestrateur/Planner : recoit la tache globale, la decompose en sous-taches et route chacune vers l'agent specialise appropriee (via function calling / tool use). Chaque **agent** dispose d'un role defini, d'un acces restreint a certains outils (recherche web, execution de code, acces base de donnees), et retourne un resultat structure a l'orchestrateur. Un **etat partage** (memoire de la session multi-agent) permet aux agents de s'appuyer sur les resultats intermediaires des autres. Une boucle de **validation/critique** peut faire relire le resultat d'un agent par un autre avant de le considerer final.

Trade-offs clés

Un seul agent generaliste vs plusieurs agents specialises ? La specialisation ameliore la qualite sur chaque sous-tache (un prompt/contexte plus cible par agent) mais multiplie le nombre d'appels LLM (cout et latence cumules) et introduit un risque de divergence/incoherence entre agents si l'orchestration est mal concue. **Comment eviter les boucles infinies ?** Imposer une limite stricte du nombre d'iterations/appels d'outils par tache, avec un mecanisme de "budget" explicite communique a l'orchestrateur.

3.7 Design ML Feature Store

Exigences

Fonctionnel : centraliser le calcul et le stockage des features utilisees pour l'entrainement et l'inference des modeles ML, garantissant que les memes features sont utilisees dans les deux contextes. **Non-fonctionnel** : servir les features en ligne avec une latence de quelques millisecondes (inference temps-reel), tout en supportant des calculs batch lourds pour l'entrainement.

Architecture haut niveau

Architecture a double chemin, le probleme central que resout un feature store : **Offline store** (data warehouse/lake) – historique complet des features, utilise pour entrainer les modeles sur de grands volumes avec du temps de calcul disponible. **Online store** (base cle-valeur basse latence, ex : Redis/DynamoDB) – derniere valeur connue de chaque feature par entite, servie a l'inference en production. Un pipeline de calcul de features (batch et/ou streaming) alimente les deux stores en parallele a partir de la meme logique de transformation, pour garantir leur coherence.

Trade-offs clés

Le probleme que le feature store resout explicitement est le **training-serving skew** : sans lui, l'equipe data science calcule les features d'entrainement en Python/SQL batch, tandis que l'equipe engineering recode independamment la meme logique pour l'inference temps-reel –

toute divergence entre les deux implementations degrade silencieusement la performance du modele en production. Centraliser la definition de la feature (une seule logique de transformation, executee pour alimenter les deux stores) elimine structurellement ce risque, au prix d'une infrastructure supplementaire a maintenir.

4. Systemes metier critiques

4.1 Design Fraud Detection System (temps-reel)

Exigences

Fonctionnel : evaluer chaque transaction en quasi temps-reel et decider de l'accepter, la bloquer ou la mettre en revue manuelle, en s'appuyant sur un modele ML et des regles metier. **Non-fonctionnel** : decision en moins de 100–200ms (ne doit pas degrader l'experience de paiement), tres faible taux de faux positifs (bloquer une transaction legitime coute cher en experience client), audit complet des decisions.

Architecture haut niveau

Transaction → **Kafka** (ingestion de l'evenement de transaction) → **Spark Streaming** (enrichissement : calcul de features contextuelles comme "nombre de transactions dans la derniere heure pour cet utilisateur", lues/mises a jour dans un **feature store Redis** pour la faible latence) → **ML Service** (scoring du risque via un modele pre-entraîne) combine a des **regles metier** explicites (ex : montant > seuil ET pays inhabituel) → decision routee : auto-approve, auto-block, ou file de **revue manuelle**. Toute decision est journalisee dans **PostgreSQL** pour l'audit, et un **dashboard/service d'alerte** notifie les analystes sur les patterns suspects agreges.

Trade-offs cles

Regles metier vs modele ML : les regles sont explicables et modifiables instantanement (utile pour reagir a une fraude emergente sans re-entraîner un modele) mais rigides et faciles a contourner une fois connues ; le ML detecte des patterns subtils non anticipes mais agit comme une boite plus ou moins noire et necessite un re-entrainement periodique face au *concept drift* (les fraudeurs adaptent leurs techniques). Les systemes de production combinent les deux : les regles filtrent les cas evidents a faible cout, le ML traite la zone grise. **Latence vs precision** : un modele plus complexe (ensemble, deep learning) ameliore potentiellement la precision mais peut violer le budget de latence – d'où l'usage frequent de features precalculees (feature store) plutot que de tout calculer a la volée.

4.2 Design Payment Gateway

Exigences

Fonctionnel : router une demande de paiement vers le bon processeur (carte, virement), gerer les etats du cycle de vie d'une transaction (initiee, autorisee, capturee, remboursee), notifier les systemes tiers via webhooks. **Non-fonctionnel** : **idempotence stricte** (ne jamais debiter deux fois pour une meme intention de paiement), conforme reglementaire (PCI-DSS), coherence forte sur l'etat financier (pas de "presque coherent" tolerable ici).

Architecture haut niveau

API de paiement (exige une *idempotency key* fournie par le client sur chaque requete de creation) → **Service de paiement** qui persiste l'intention de transaction dans une base **SQL** avec des transactions ACID avant tout appel externe → appel au **processeur de paiement externe** (Stripe, reseau bancaire) → mise a jour de l'etat en base selon la reponse, avec une **machine a etats explicite** (pending → authorized → captured → settled / failed / refunded)

→ **webhooks** vers les systemes consommateurs (avec retry et signature de verification) pour notifier le resultat de maniere asynchrone et fiable.

Trade-offs clés

Pourquoi une base SQL et non NoSQL ici ? L'état financier exige des transactions ACID et une coherence forte immediate – c'est le cas d'usage canonique ou SQL est le choix par default, contrairement a un feed social ou l'eventual consistency est acceptable. **Idempotency key** : le client genere une cle unique par intention de paiement ; si la requete est retentee (timeout reseau, retry client), le serveur reconnait la cle deja traitee et retourne le meme resultat sans debiter une seconde fois – protection essentielle contre le double-paiement en cas d'echec reseau ambigu (le client ne sait pas si sa premiere requete a reussi).

4.3 Design Uber Dispatch / Ride Sharing Backend

Exigences

Fonctionnel : mettre en relation un passager avec le chauffeur disponible le plus pertinent (proximite, ETA), suivre la position en temps-reel pendant la course, calculer un prix dynamique. **Non-fonctionnel** : latence de matching de quelques secondes, mises a jour de position a haute frequence pour des millions de vehicules simultanes, disponibilite critique (un service indisponible = perte de revenu immediate).

Architecture haut niveau

Ingestion de position : chaque vehicule envoie sa position via une connexion persistante (WebSocket) a intervalle regulier → ecrite dans un **index geospatial** (ex : geohashing ou quadtree, permettant de requeter rapidement "tous les chauffeurs dans un rayon de 2km") maintenu en memoire (Redis avec extension geospatiale, ou service dedie). **Service de matching** : a la demande d'un passager, interroge l'index geospatial pour les candidats proches, applique un algorithme de scoring (distance, ETA, note du chauffeur) et propose la course au meilleur candidat avec un court delai d'acceptation avant de proposer au suivant. **Service de pricing** : calcule le prix en fonction de l'offre/demande locale en temps-reel (ratio demandes/chauffeurs disponibles par zone).

Trade-offs clés

Index geospatial en memoire vs base geospatiale classique : la frequence de mise a jour des positions (chaque vehicule toutes les quelques secondes, pour des millions de vehicules) impose une structure optimisee pour des ecritures/lectures tres rapides en memoire plutot qu'une base disque classique, meme avec index spatial – au prix d'une durabilite moindre (acceptable ici car une position perimee de quelques secondes est simplement remplacee par la suivante). **Matching glouton vs matching global optimise** : proposer immediatement au chauffeur le plus proche est simple et rapide mais localement sous-optimal a l'echelle d'une zone dense (deux passagers proches pourraient echanger leurs chauffeurs pour un ETA global meilleur) – les systemes avances batchent le matching sur de courtes fenetres pour optimiser globalement, au prix d'une latence de matching legerement accrue.

4.4 Design Netflix Streaming (video a la demande)

Exigences

Fonctionnel : parcourir un catalogue, lancer la lecture d'une video avec demarrage quasi instantane, adapter la qualite au reseau disponible sans interruption. **Non-fonctionnel** : tres faible latence de demarrage de lecture partout dans le monde, tolerance aux variations de bande passante, cout de bande passante maitrise a l'echelle de centaines de millions d'utilisateurs.

Architecture haut niveau

Pipeline d'ingestion/encodage (hors chemin critique utilisateur) : une video source est encodee en *plusieurs resolutions et bitrates*, decoupee en petits segments (quelques secondes chacun), selon un protocole de streaming adaptatif (HLS/DASH). **Distribution** : ces segments sont propages vers un **CDN** (souvent un CDN propriétaire co-localise chez les FAI pour Netflix) au plus pres des utilisateurs. **Lecture client** : le lecteur video mesure en continu la bande passante disponible et choisit dynamiquement, segment par segment, la resolution la plus adaptee (*Adaptive Bitrate Streaming*) – capable de degrader la qualite sans interrompre la lecture si le reseau se degrade.

Trade-offs clés

Pourquoi decouper en segments plutot que streamer un seul flux continu ? Le decoupage permet de changer de qualite *entre deux segments* sans interrompre la lecture, et permet au CDN de cacher/servir independamment chaque segment (meilleure efficacite de cache que streamer depuis une position arbitraire dans un flux monolithique). **Encodage multi-bitrate a l'avance vs transcodage a la volée** : encoder toutes les variantes a l'avance (une fois, hors ligne) coute du stockage supplementaire mais garantit une latence de lecture minimale ; transcoder a la demande economiserait du stockage mais introduirait une latence inacceptable pour un usage a tres grande echelle – Netflix choisit massivement le pre-encodage.

4.5 Design Distributed Cache (type Redis)

Exigences

Fonctionnel : stocker des paires cle-valeur avec acces en quelques millisecondes, expiration automatique (TTL), operations atomiques simples (increment, ajout a une liste/set). **Non-fonctionnel** : tres haute disponibilite, scalabilite horizontale du volume de donnees et du debit, tolerance a la perte occasionnelle de donnees cachees (elles peuvent generalement etre recalculées depuis la source de verite).

Architecture haut niveau

Donnees en memoire (RAM) reparties sur plusieurs noeuds via **sharding** (hashing consistant pour minimiser la redistribution lors de l'ajout/retrait d'un noeud). Chaque shard peut etre replet (primaire + replicas) pour la disponibilite en lecture et la tolerance aux pannes. Politique d'**eviction** (LRU le plus souvent) declenchee quand la memoire disponible est atteinte, combinee a une expiration active par TTL.

Trade-offs clés

Hashing consistant : un hashing simple ($\text{hash}(\text{key}) \bmod N$) redistribuerait presque toutes les clés lors de l'ajout/retrait d'un nœud (cache quasi entièrement invalide) ; le hashing consistant ne redistribue qu'une fraction proportionnelle au changement de capacité, préservant l'essentiel de l'efficacité du cache pendant un scaling. **Cache-aside vs write-through** : en *cache-aside*, l'application lit le cache, et en cas d'absence (miss) va chercher en base puis peuple le cache – simple, mais le cache peut être temporairement absent/perime. En *write-through*, chaque écriture met à jour simultanément le cache et la base – le cache est toujours à jour mais chaque écriture est plus lente et plus complexe à rendre atomique entre les deux stores.

5. Produits grand public a forte echelle

5.1 Design URL Shortener (TinyURL / Bitly)

Exigences

Fonctionnel : raccourcir une URL longue en une URL courte unique, rediriger vers l'URL d'origine lors de l'accès. **Non-fonctionnel** : redirection quasi instantanee (lecture tres dominante par rapport a l'écriture, ratio souvent 100 :1 ou plus), disponibilite elevee (une panne bloque toutes les redirections existantes), URLs courtes non predictibles/collisionnables.

Estimation (back-of-envelope)

Charge dominee par les lectures : pour 100M nouvelles URLs/mois (≈ 40 écritures/sec), un ratio lecture/écriture de 100 :1 donne $\approx 4\,000$ lectures/sec a supporter – oriente naturellement vers une architecture fortement cachee.

Architecture haut niveau

Écriture : generer un identifiant court unique (encodage base62 d'un compteur distribue, ou hash tronque de l'URL avec gestion de collision) → stocker le mapping **code court** → URL **longue** dans une base **cle-valeur** (tres bonne adequation au modele d'accès simple). Lecture : **cache** (Redis) devant la base pour absorber l'immense majorite des redirections sans toucher au stockage persistant, avec redirection HTTP 301/302 vers l'URL d'origine.

Trade-offs cles

Compteur distribue vs hash : un compteur global (ranges pre-alloues par noeud pour eviter un point de contention) garantit l'unicite sans collision et produit des codes courts predictibles en longueur ; un hash (ex : MD5 tronque) evite la coordination d'un compteur mais impose de gerer les collisions (rares mais possibles). **301 vs 302** : 301 (redirection permanente) est mis en cache par le navigateur/CDN, reduisant la charge serveur sur les acces repetes mais empechant de changer la destination ou de suivre les clics ulterieurs ; 302 (temporaire) permet le tracking d'analytics a chaque clic au prix d'une charge serveur plus elevee.

5.2 Design Rate Limiter

Exigences

Fonctionnel : limiter le nombre de requetes qu'un client (utilisateur, IP, cle API) peut effectuer dans une fenetre de temps donnee, rejeter les requetes en excès. **Non-fonctionnel** : decision de limitation en quelques millisecondes sans devenir elle-meme un goulot d'etranglement, coherence du compteur meme derriere plusieurs instances de service en parallele.

Architecture haut niveau

Un compteur par client, stocke dans un **cache distribue** (Redis, pour la faible latence et le partage d'etat entre toutes les instances du service). Chaque requete entrante incremente atomiquement (INCR Redis) le compteur associe a sa fenetre de temps courante ; si le compteur depasse le seuil autorise, la requete est rejetee (HTTP 429). Le rate limiter est generalement implemente comme middleware/filtre en amont de l'API, avant tout traitement metier.

Trade-offs clés

Algorithmes de limitation : *Fixed Window* (compteur remis à zéro à intervalle fixe – simple mais autorise un pic de 2x le seuil à la frontière de deux fenêtres) ; *Sliding Window Log* (historique précis de chaque requête – exact mais coûteux en mémoire) ; *Sliding Window Counter* (approxime le sliding window en pondérant deux fenêtres fixes – bon compromis précision/cout, choix fréquent en production) ; *Token Bucket* (des jetons se rechargent à taux constant, chaque requête consomme un jeton – autorise naturellement des rafales courtes tout en bornant le débit moyen, très utilisé pour les API publiques).

5.3 Design Notification Service

Exigences

Fonctionnel : envoyer des notifications (push mobile, email, SMS) déclenchées par des événements applicatifs, avec priorisation et gestion des préférences utilisateur. **Non-fonctionnel** : haut débit d'envoi en période de pic, pas de duplication de notification, dégradation gracieuse si un provider externe (APNS, FCM, SendGrid) est indisponible.

Architecture haut niveau

Service producteur d'événement → **file de messages** (Kafka/SQS) pour découpler la génération de l'envoi et absorber les pics → **workers de notification** consomment la file, vérifient les préférences utilisateur (opt-in/opt-out par canal) et le *throttling* (éviter de spammer), puis appellent le **provider externe** approprié selon le canal (APNS/FCM pour push, SendGrid/SES pour email, Twilio pour SMS). Un **service de templates** centralise le contenu des notifications pour éviter la duplication de logique de formatage.

Trade-offs clés

Pourquoi une file de messages plutôt qu'un appel direct au provider ? Découple le pic de génération d'événements (ex : une alerte déclenchée pour des millions d'utilisateurs simultanément) du débit que les providers externes peuvent effectivement absorber – la file lisse la charge et permet des retries sans bloquer le service émetteur. **Idempotence** : chaque notification porte un identifiant unique ; un worker qui retente après un échec ambigu (timeout du provider sans confirmation) vérifie d'abord si la notification a déjà été envoyée, évitant une double notification désagréable pour l'utilisateur.

5.4 Design WhatsApp / Messenger (messaging instantané)

Exigences

Fonctionnel : envoyer/recevoir des messages en quasi temps-reel en tête-à-tête et en groupe, indicateurs de statut (envoyé/reçu/lu), fonctionnement hors-ligne avec livraison différée. **Non-fonctionnel** : latence de livraison de l'ordre de la centaine de millisecondes, garantie de livraison (pas de message perdu), scalabilité à des milliards de messages/jour, chiffrement de bout en bout.

Architecture haut niveau

Connexions persistantes : chaque client maintient une connexion **WebSocket** avec un serveur de messagerie, permettant au serveur de *pousser* les messages entrants sans polling. **Routage** : un **service de presence** (souvent adosse a Redis) maintient quel serveur gere la connexion active de chaque utilisateur ; a l'envoi d'un message, le serveur destinataire est identifie et le message lui est route (via un bus interne) pour livraison immediate si l'utilisateur est en ligne. **Persistance et hors-ligne** : chaque message est d'abord ecrit durablement (base optimisee en ecriture, souvent NoSQL partitionnee par conversation) avant tentative de livraison – si le destinataire est hors-ligne, le message reste en attente et sera livre/pousse (via notification push) a sa reconnexion.

Trade-offs cles

WebSocket vs polling : le polling repete impose une latence liee a l'intervalle de polling et gaspille des ressources sur des verifications sans nouveaute ; WebSocket maintient une connexion ouverte permettant une livraison pousse quasi instantanee, au prix de devoir gerer un tres grand nombre de connexions longue-duree cote serveur (dimensionnement different d'un service HTTP stateless classique). **Ecriture avant livraison** : ecrire le message en base *avant* de tenter la livraison garantit qu'aucun message n'est perdu meme si la tentative de livraison temps-reel echoue – le hors-ligne devient un cas normal traite par le meme mecanisme (relecture des messages non-acquittes a la reconnexion) plutot qu'un cas d'erreur special.

5.5 Design Instagram Feed / Facebook News Feed

Exigences

Fonctionnel : afficher a chaque utilisateur un flux de publications de ses abonnements, classe par pertinence/recence. **Non-fonctionnel** : chargement du feed en quelques centaines de millisecondes, gestion du cas extreme d'un compte avec des dizaines de millions d'abonnes ("fan-out" massif), fraicheur raisonnable (nouveau post visible rapidement par les abonnes actifs).

Architecture haut niveau

Deux strategies combinees selon le profil : **Fan-out on write (push)** – a la publication d'un post, il est immediatement insere dans le feed precalcule (cache) de chaque abonne ; le chargement du feed devient une simple lecture rapide. **Fan-out on read (pull)** – pour les comptes a tres grand nombre d'abonnes (celebrities), precalculer le push vers des millions de feeds serait trop couteux ; le feed de leurs abonnes est alors assemble a la lecture en fusionnant les posts precalcules (comptes normaux) avec les posts recents des comptes a forte audience recuperes a la demande.

Trade-offs cles

C'est le trade-off fan-out le plus classique en system design : **push** optimise la lecture (l'operation la plus frequente, largement dominante) au prix d'ecritures amplifiees a la publication (un post d'un compte a 10M abonnes = 10M ecritures) ; **pull** evite cette amplification a l'ecriture mais alourdit chaque lecture de fusion en temps-reel. La solution hybride (push par default, pull pour les comptes a tres large audience au-dela d'un seuil) capture le meilleur des deux, au prix d'une complexite d'implementation superieure (deux chemins de code a maintenir).

5.6 Design YouTube (plateforme video)

Exigences

Fonctionnel : upload de video, encodage en plusieurs qualites, lecture en streaming adaptatif, recherche et recommandations. **Non-fonctionnel** : traitement d'upload asynchrone et resilient (des videos volumineuses, l'encodage prend du temps), distribution mondiale a faible latence, scalabilite du stockage a l'echelle de l'exaoctet.

Architecture haut niveau

Upload : le fichier video est envoye vers un stockage objet temporaire, l'upload confirme immediatement a l'utilisateur → un evenement declenche de maniere asynchrone un **pipeline d'encodage** (souvent distribue, decoupant la video en segments codes en parallele sur plusieurs workers) produisant les variantes multi-resolution. **Stockage final** : les segments codes sont ecrits en stockage objet durable puis distribues via **CDN**. **Metadonnees** (titre, description, statistiques de vues) dans une base separee du contenu binaire lui-meme. **Lecture** : streaming adaptatif comme pour Netflix (voir section precedente), avec le compteur de vues incremente de maniere asynchrone (pas sur le chemin critique de lecture) pour eviter d'ecrire en base a chaque seconde de visionnage.

Trade-offs clés

Traitement synchrone vs asynchrone de l'upload : encoder une video en plusieurs resolutions avant de confirmer l'upload bloquerait l'utilisateur potentiellement plusieurs minutes ; traiter l'encodage de maniere asynchrone (l'utilisateur voit "video en cours de traitement") ameliore drastiquement l'experience percue au prix d'un delai avant disponibilite reelle de la video. **Separation metadonnees/contenu binaire** : les metadonnees (forte frequence de lecture/ecriture, petite taille) et le contenu video (tres volumineux, majoritairement en lecture via CDN) ont des profils d'accès radicalement differents justifiant des systemes de stockage distincts et independamment scalables.

5.7 Design Google Drive / Dropbox (stockage et synchronisation de fichiers)

Exigences

Fonctionnel : upload/download de fichiers, synchronisation automatique entre plusieurs appareils, partage avec controle d'accès, historique de versions. **Non-fonctionnel** : coherence eventuelle acceptable entre appareils (quelques secondes de decalage tolerable), efficacite reseau (ne pas re-transférer un fichier entier pour un petit changement), durabilite tres forte des donnees (perte de fichier utilisateur inacceptable).

Architecture haut niveau

Chunking : chaque fichier est decoupe en blocs (chunks) de taille fixe, chacun identifie par un hash de son contenu. **Stockage** : les chunks sont stockes dans un stockage objet durable (replique/erasure-coded pour la durabilite), dedupliques naturellement si deux fichiers partagent des chunks identiques. **Metadonnees** : une base separee associe chaque fichier a la liste ordonnee de ses chunks, plus les informations de version et de permissions. **Synchronisation** : un client local detecte les modifications (souvent via des evenements systeme de fichiers), ne transfere que les chunks modifies/nouveaux (pas le fichier entier), et un service de notification (souvent via une connexion persistante) informe les autres appareils du meme

utilisateur qu'une mise a jour est disponible pour declencher leur propre synchronisation.

Trade-offs clés

Chunking : decouper en blocs permet de ne transferer que les portions modifiees d'un gros fichier (efficacite reseau majeure pour des fichiers volumineux edites incrementalement) et permet une deduplication naturelle au niveau du stockage. Compromis : la reconstruction d'un fichier necessite de recuperer et reassembler tous ses chunks, ajoutant une complexite absente d'un stockage "fichier entier". **Resolution de conflits** : si le meme fichier est modifie hors-ligne sur deux appareils avant resynchronisation, le systeme doit detecter le conflit (via versioning/vecteurs d'horloge) et generer une copie de conflit plutot que silencieusement ecraser une des deux versions.

5.8 Design Reddit (forum de discussion avec vote)

Exigences

Fonctionnel : publier des posts dans des communautes (subreddits), voter (upvote/downvote), classer les posts par score/recence, commenter en fils imbriques. **Non-fonctionnel** : lecture tres dominante par rapport a l'ecriture, calcul de classement (ranking) a jour sans recalculer sur l'integralite des posts a chaque requete, gestion de fils de commentaires profondement imbriques.

Architecture haut niveau

Ecriture d'un vote : incremente/decremente un compteur associe au post (dans un cache/compteur distribue pour absorber le volume de votes sans surcharger la base principale), la mise a jour du *score de classement* (souvent une combinaison du nombre de votes et de la recence, decroissant avec le temps) est recalculee de maniere incrementale, pas recalculee globalement. **Lecture du feed d'un subreddit** : les posts sont precalcules/caches, tries par le score de classement maintenu en quasi temps-reel, avec pagination. **Commentaires** : stockes avec une reference a leur parent, permettant de reconstruire l'arbre d'imbrication a la lecture – souvent charges de maniere paresseuse (les reponses profondement imbriquees ne sont charges qu'a la demande de l'utilisateur).

Trade-offs clés

Recalcul incremental vs global du score : recalculer le classement de tous les posts d'un subreddit populaire a chaque vote serait bien trop couteux ; maintenir un score incrementalement mis a jour (formule de type "hot ranking" combinant votes et decroissance temporelle) permet un tri efficace en base/cache sans recalcul de masse. **Stockage des commentaires en arbre** : stocker chaque commentaire avec juste une reference a son parent (plutot que d'imbriquer physiquement les structures) simplifie l'ecriture d'un nouveau commentaire (toujours une simple insertion) au prix d'une reconstruction de l'arbre necessaire a la lecture.

5.9 Design Slack / Discord (messagerie d'équipe et communautés)

Exigences

Fonctionnel : canaux de discussion textuelle en temps-reel, historique consultable, presence en ligne, notifications push pour les mentions. **Non-fonctionnel** : livraison en temps-reel a de nombreux membres simultanés d'un meme canal (contrairement a WhatsApp, un canal peut avoir des milliers de membres actifs), recherche full-text performante dans un historique volumineux.

Architecture haut niveau

Similaire a l'architecture de messagerie instantanée (connexions **WebSocket** persistantes, service de presence), avec une difference structurelle cle : un message envoyé dans un canal doit être diffusé (*fan-out*) a tous les membres actuellement connectés a ce canal, pas a un seul destinataire – gère via un mécanisme de *pub/sub* interne (le serveur qui reçoit le message le publie sur un canal interne, chaque connexion WebSocket abonnée au canal correspondant le reçoit et le pousse a son client). L'historique est indexé dans un moteur de recherche full-text (Elasticsearch) en parallèle du stockage principal, pour la fonctionnalité de recherche dans les messages passés.

Trade-offs clés

Fan-out par canal vs par utilisateur : diffuser a tous les membres connectés d'un canal (plutôt que de dupliquer le message dans une file par destinataire comme pour de la messagerie 1-a-1) évite l'amplification d'écriture sur les canaux très peuplés, mais impose une architecture de *pub/sub* interne capable de router efficacement vers potentiellement des milliers de connexions WebSocket simultanées réparties sur plusieurs serveurs. **Recherche full-text séparée** : indexer dans un moteur dédié (plutôt que `LIKE '%terme%'` sur la base principale) est nécessaire dès que le volume d'historique et les besoins de pertinence de recherche dépassent ce qu'une base relationnelle généraliste gère efficacement.

6. Aide-memoire final

A retenir Les patterns qui reviennent partout

Malgre la diversite des 30 sujets ci-dessus, un petit nombre de patterns architecturaux recurrents suffit a couvrir la grande majorite des cas : **fan-out on write vs on read** (feed, notifications), **sharding + hashing consistant** (cache, base de donnees, index geospatial), **decouplage par file de messages** (Kafka partout ou producteur et consommateur ont des debits/disponibilites differents), **cache devant une base** (lecture dominante), **CDC/log-based capture** (propager un changement sans re-interroger la source), et **architecture en deux etages retrieval + ranking** (recherche, recommandation, RAG). Reconnaître lequel de ces patterns s'applique a un nouveau probleme est plus utile que de memoriser 30 architectures distinctes.

Rappel : le CAP theorem en pratique

Face a une partition reseau, un systeme distribue doit choisir entre **Consistency** (toutes les lectures voient la derniere ecriture) et **Availability** (le systeme continue de repondre). En pratique, la plupart des systemes visent une *eventual consistency* (disponible, coherent apres un court delai) pour les donnees a faible enjeu (feed social, compteur de vues), et une coherence forte uniquement la ou l'enjeu metier l'exige explicitement (solde bancaire, inventaire, paiement).